ChatGPT

Data Curation at Scale: Quality, Filtering, Deduplication, and Documentation

Introduction

Modern machine learning models, especially large language models (LLMs), rely on massive datasets that are often sourced from web crawls and other large-scale repositories. Curating such datasets at scale presents unique challenges in ensuring **data quality**, removing **undesirable or low-quality content**, eliminating **duplicate entries**, and properly **documenting** the dataset for transparency and accountability. This study guide covers conceptual foundations and practical techniques for data curation at scale, including web-crawl data pipelines, heuristic and model-based filtering methods, scalable deduplication (e.g. using MinHash/SimHash), and best practices for dataset documentation (e.g. "datasheets" and "data statements"). We assume the reader has a background in machine learning and data engineering, and we focus on high-level understanding, case studies, and key literature in this domain.

Why Data Curation Matters: The quality of a dataset fundamentally influences the performance and behavior of models trained on it [1] [2]. Poorly curated data can introduce noise, biases, and even legal or ethical risks. Large "foundational" models trained on uncurated web data might end up learning and reproducing unwanted societal biases or private information present in the data [3] [4]. At the same time, due to the sheer scale of data (often terabytes of text or more), manual inspection and cleaning is infeasible, necessitating automated curation pipelines. Effective data curation at scale can **improve model performance and efficiency** while **mitigating risks** associated with bad data. For example, one study found that thoroughly deduplicating a large text corpus (Common Crawl's C4) reduced the frequency of a language model regurgitating memorized training data by **10×** and eliminated a significant portion of train–test overlap, all with *no loss in model accuracy* [4]. In fact, deduplication allowed reaching the same accuracy with fewer training steps, highlighting how cleaner data can yield more efficient training [5].

In the following sections, we will explore the stages of a typical web-scale data curation pipeline: from raw data acquisition to filtering and deduplication, and finally documentation. We'll illustrate concepts with examples (including a simple MinHash deduplication example and real-world case studies like the Common Crawl–based C4 dataset and the RefinedWeb dataset for Falcon LLM). We'll also reference key literature such as **Gebru et al. (2018)** on "datasheets for datasets" [2], **Bender & Friedman (2018)** on "data statements" for documenting datasets [6], **Lee et al. (2021)** on the importance of deduplicating training data [4], and **Penedo et al. (2023, 2024)** on recent large-scale web data curation pipelines (RefinedWeb and FineWeb) [7] [8]. By the end of this guide, you should have a solid conceptual understanding of how to curate data at scale for machine learning, why each step is necessary, and how to document the outcome for the benefit of the community.

Overview of Web-Scale Data Curation Pipelines

A **data curation pipeline** is a sequence of processing steps that transform raw data (often noisy and unstructured) into a cleaned, structured dataset ready for model training. At web-scale, pipelines must

handle billions of documents and therefore emphasize efficiency and scalability. A canonical example is the pipeline used to create the **C4 (Colossal Cleaned Common Crawl) dataset** for training Google's T5 model, which involved downloading Common Crawl web snapshots, extracting text, filtering out undesirable content, and deduplicating texts [9]. Similarly, Meta's **CCNet pipeline** (used as a basis for LLaMA's data) delineates clear stages such as deduplication, language identification, content filtering, etc [10].

Key Stages in a Web-Crawl Data Pipeline:

- **Data Acquisition:** Obtain raw data, often from a **web crawl dump** like Common Crawl. Common Crawl provides data in formats such as WARC (raw HTML responses), WAT (metadata and links), and WET (extracted plaintext) [11] [12]. A pipeline may start with WET files (plaintext) for simplicity, or use WARC/WAT with custom text extraction tools for potentially better quality text [13]. For example, *The Pile* dataset used a custom extractor (jusText) on raw web data instead of relying on WET, to obtain cleaner text [14], while *RefinedWeb* (Falcon LLM's dataset) similarly processed WARC with the Trafilatura library to extract text [15].

- **URL and Domain Filtering:** Before processing the content, many pipelines perform coarse filtering at the URL or domain level. This involves removing any pages from known spam domains, pornographic or violent content sites, malware sources, etc., by checking the URL against large blocklists. For instance, the RefinedWeb pipeline applied a blocklist of **4.6 million domains** (aggregated from multiple sources) to filter out undesirable websites, and also dropped pages if the URL contained certain blacklisted keywords [16]. This step keeps out chunks of the crawl that are unlikely to yield useful training data or could be problematic content.

- **Text Extraction and Cleanup:** Convert raw web pages (HTML) into plain text. If using WET files, the text is pre-extracted, but some pipelines prefer to extract text themselves to have more control (as mentioned above). The extraction step usually includes removing HTML tags, script/style content, navigation menus, and other boilerplate. Libraries like *jusText*, *BeautifulSoup*, or *Trafilatura* are commonly used. The output is textual documents (often one document per webpage) which can then be further cleaned. This cleanup might involve stripping extraneous whitespace, normalizing Unicode characters, lowercasing (depending on pipeline), etc. A notable point is that extraction quality matters: better extraction can omit clutter and yield more coherent text content for the dataset [14].

- **Document Splitting/Sharding:** At massive scale, it's helpful to break the data into smaller shards for parallel processing. For example, CCNet divided Common Crawl data into 5GB JSON shards (each containing thousands of documents) to manage memory and parallelize deduplication and filtering operations [17]. Sharding also allows certain operations (like deduping within and across shards) to be done more manageably.

- **Deduplication (Initial Pass):** It's common to perform an initial **deduplication** early in the pipeline, often at a fine granularity (paragraph or line level), to remove boilerplate and exact duplicates. CCNet normalized each paragraph (lowercase, removed punctuation, etc.) and computed a hash (SHA-1) – if two paragraphs had the same hash, one was dropped [18] [19]. This process, done at paragraph level, eliminated a huge amount of redundancy – *around 70% of the raw text was identified as duplicate boilerplate in CCNet* [20]! Removing these repetitive chunks (like repeated headers, footers, cookie notices, template text, etc.) not only reduces dataset size but also **improves quality**

by removing content that isn't meaningful natural language. After this step, the data is much leaner and easier to handle for language filtering and other steps. *(We will discuss more on deduplication techniques in a dedicated section below.)*

- **Language Identification & Filtering:** Large web crawls are multilingual. Using a language identification (LID) tool (like FastText's language classifier [21]) allows tagging each document with a predicted language (and a confidence score). The pipeline can then either keep only a target language (e.g. only English for an English dataset) or separate the corpus by language. Typically, a confidence threshold is applied – e.g., CCNet kept a page if the top language had >50% confidence [22], whereas RefinedWeb, aiming for higher precision, used a 65% threshold [23]. (RefinedWeb also interestingly did LID **before** deduplication, whereas CCNet did dedup first; pipeline order can vary [23].) In practice, you might maintain multilingual data but ensure language labels are recorded, or filter out low-confidence language detections to remove gibberish. Ensuring the dataset has the intended language content is crucial for training a model that performs well in that language.

- **Content Filtering (Quality Filtering):** This stage removes *undesirable or low-quality content* from the text. Here is where a combination of **heuristic filters** and **model-based filters** come into play (we explore each in detail in the next section). In summary, heuristic (rule-based) filters might drop documents that are too short or too long, have too high a fraction of weird characters or boilerplate, or contain prohibited words (hate speech, personal data like SSNs, etc.). Model-based filters might use a classifier or language model to score each document on quality metrics (perplexity, likelihood of being natural text, toxicity score, etc.), then remove those below a threshold. For example, **CCNet** used **language model perplexity** as a proxy for quality: they trained 5-gram language models on Wikipedia text for each language, and for each candidate document computed its perplexity under that model [24]. Documents with extremely high perplexity (relative to typical Wikipedia perplexity) are likely to be incoherent or unrelated text (e.g., random content, code, or very niche jargon) – these were considered low quality. However, CCNet's authors decided not to automatically remove the "high perplexity" tail entirely, noting that some of those outliers (like forum slang or technical jargon) could be useful for certain applications [25]. In contrast, **RefinedWeb** avoided using a language model for filtering; instead it relied on simpler heuristic rules – filtering out documents that were outliers in terms of length, the ratio of symbols-to-words, etc., to ensure the content is actual natural language [26]. Such heuristics, inspired by DeepMind's **Gopher** pipeline, were applied per-language (since, for example, what counts as an "odd" character ratio might differ between writing systems) [27]. Other pipelines might incorporate **blacklists/whitelists** of content: e.g., removing documents containing known profanity or disallowed content, or conversely ensuring certain high-quality sources are whitelisted. Another advanced filtering step (used in LLaMA's data) was an "**is-reference**" classifier: LLaMA's team trained a linear classifier to distinguish Wikipedia reference-text pages versus random web pages, and then *removed any page classified as non-reference* [28]. The idea was to filter out pages that were less likely to be informative content. This shows an example of a learned model filter beyond just perplexity or toxic content filtering.

- **Final Deduplication (Near-Duplicate Removal):** After filtering, a second round of deduplication is often performed to catch *near-duplicates* or any remaining redundancy, this time at the document level. The initial dedup (paragraph-level) handled exact duplicates and boilerplate; now we want to ensure we don't have highly similar documents (which could be, say, the same article from mirror sites, or pages with overlapping content). Techniques like **MinHash** and **SimHash** (or more generally, *Locality-Sensitive Hashing* methods) are employed to efficiently detect and remove near-duplicate

documents at scale. We dedicate a full section to these methods later, including an illustrative example of MinHash. As a case study, the **RefinedWeb** project applied a fuzzy deduplication with MinHash after removing exact duplicates, and they used "strict settings" that removed a **very high fraction** of data – significantly higher than earlier datasets – because so much spammy replicated content (like SEO boilerplate across many sites) was identified and pruned [20] [29]. This aggressive deduplication greatly reduced the raw 5 trillion token corpus to a smaller high-quality set, albeit at a possible cost of losing some diversity [20].

- **Assembly and Splits:** The cleaned, filtered, deduplicated documents are then assembled into the final dataset. This may involve concatenating or shuffling data, and splitting into training/validation/test sets if needed. Care must be taken that no near-duplicates exist across the splits (to prevent *contamination*, where test data is seen in training). Many projects explicitly ensure no overlap between train and validation by checking hashes or n-grams across splits [30] [31]. For example, GPT-3's team filtered out any training example that had a 13-gram overlap with any evaluation set example, to avoid contaminating evaluation [32].

- **Documentation:** Finally, the curators prepare documentation describing the dataset's creation, characteristics, and recommended uses. This often takes the form of a **datasheet** or **data statement** (discussed later in detail). Documentation is crucial for transparency – it informs anyone using the dataset about its provenance, composition, known biases or limitations, and the filtering steps applied to obtain it [2] [33]. It can also include summary statistics (e.g. number of tokens, language breakdown, topic/domain breakdown, etc.) and any ethical considerations.

Below is an illustration of a typical pipeline structure, based on the CCNet/LLaMA process:

Figure 1: High-level overview of a web-crawl data curation pipeline (from CommonCrawl to cleaned data). Key stages include deduplication (to remove boilerplate and duplicates), language identification, content filtering (heuristic and model-based), and more [10]. Each stage incrementally refines the dataset's quality.

Each project might implement these steps slightly differently or in different order, but the components above are commonly found across most web-scale text dataset curation efforts. In the next sections, we will delve deeper into the **quality filtering techniques** (heuristic vs. model-based) and **deduplication methods** (with a focus on MinHash/SimHash for near-duplicate detection), followed by examples and case studies illustrating these in practice.

Data Quality Filtering Techniques

One of the biggest challenges in data curation is defining and detecting "quality." Web data is noisy: it contains everything from well-written articles and literature to forum rants, spam blog comments, code dumps, or random strings. For training robust ML models, especially in NLP, we typically want **natural language content that is coherent, meaningful, and representative** of the domains we care about, while avoiding harmful content (hate speech, etc.) and garbage (random or low-information text). Data quality filtering thus involves a mix of **heuristic rules** and **model-based approaches** to automatically flag and remove undesirable data.

Heuristic (Rule-Based) Filtering

Heuristic filtering uses manually designed rules or thresholds based on intuitive proxies for quality. These are often straightforward to implement and fast to run, making them attractive for initial passes over huge datasets. Some common heuristic filters include:

- **Length and Character Composition Filters:** Remove extremely short documents (e.g., those with only a few characters or words) since they often aren't useful (a single-word page, or an empty page). Also remove overly long texts that exceed practical length (though in web crawls, very long texts are rarer except maybe huge code dumps or base64 blobs). Additionally, filter documents with abnormal character distributions – e.g., if a page is 90% punctuation or digits (which might indicate it's mostly code or a list of numbers), or if there's a very high ratio of non-alphanumeric symbols. Such rules help weed out content that isn't natural sentences or paragraphs (e.g., HTML or JSON blobs that text extraction missed, or pages of mostly code). The RefinedWeb pipeline, for instance, applied criteria like maximum allowed symbol-to-word ratio and ignored texts with excessive repetition of the same line or character, to ensure the text is natural language [26] [29].

- **Boilerplate and Template Detection:** Even after HTML tag removal, some boilerplate text might persist. For example, a template "About us" or navigation menu that appears on every page of a site. Heuristics can detect these by spotting *repeated substrings* or common patterns (e.g., if the same 100-character string appears in hundreds of documents, it's likely boilerplate and can be excised or those documents dropped). Tools like *boilerplate removal* libraries use such techniques. In CCNet's approach, much boilerplate was automatically removed by the paragraph-level deduplication (as identical paragraphs repeated across pages) [19]. Alternatively, one could maintain a list of common boilerplate terms (e.g., "Copyright © 2023 [SiteName]") to strip out.

- **Language Mismatch and Encoding Issues:** Sometimes a document might be misclassified or contain large chunks in a different language or gibberish (from encoding errors). Heuristics like "if a supposed English document contains >30% unrecognized characters or foreign script characters, drop it" can be applied. Also, if language ID is uncertain (low confidence) or indicates the content is in a language you're not collecting, the document can be removed or routed to another language's dataset.

- **Profanity/Hate Speech Filters:** Many datasets apply rudimentary content moderation via keyword blacklists. For example, any document containing very strong profanity or racial slurs might be removed to reduce harmful content in the training data. While this approach can catch blatant cases, it has to be used carefully (a naive keyword list can accidentally remove innocuous content, e.g., the word "sex" appears in biology or gender discussions). More sophisticated versions might use **regular expressions** or contextual cues (but those verge on model-based filtering if context is considered). As an example, the C4 dataset was known to use a list of "bad words" to filter out documents; however, this led to unintended consequences like filtering out pages about programming in "JavaScript" (because the substring "JavaScript" contains "avasc", which was a substring of an offensive term on the blocklist) [31]. This highlights that purely heuristic approaches can have false positives.

- **HTML/Formatting Artifacts:** If any documents still contain leftover HTML entities or excessive markup (like lots of `<` and `>` or ` `), they might be considered lower quality. A heuristic could be "if more than X% of characters are HTML tags or entities, skip the document".

- **Duplicate or Near-Duplicate Removal:** While deduplication is often considered a separate stage (discussed later), even simple heuristics can catch duplicates. For instance, if two documents have an identical title or if one is an exact substring of another, a rule-based approach could drop one. However, thorough near-duplicate detection is better handled by algorithms like MinHash, which we'll cover.

Heuristic filters are generally **precision-oriented** – they can precisely define certain undesired patterns (like "mostly numbers" or "contains blacklisted word") and eliminate those. They are easy to tweak (just change a threshold or add a rule) without retraining models. However, they may have **recall issues** (not catching all bad content) and can sometimes be *overly aggressive*, removing valid data that just happens to trigger a rule. It's a balancing act: for example, filtering out all text with high perplexity might remove domain-specific content that's actually valuable [25], whereas keeping it might degrade average quality.

In practice, heuristic filtering is often the first line of defense because it's computationally cheap. Modern pipelines (e.g., **GLaM, Gopher, or OpenAI's filters**) include many such rules. The **BigScience project (ROOTS corpus)** also developed an extensive set of heuristics for legal compliance (removing personal data, etc.) and quality, combined with human review for certain edge cases. A possible workflow is to apply a broad set of heuristic filters to drastically reduce the data (removing obvious junk), then use more compute-intensive model-based filters on the remainder.

Common Heuristic Filters Summary: *(to recap, here is a list of typical heuristic filtering criteria used in large-scale text corpora)*

- Remove documents that are too short (e.g. < 5 words) or that consist mostly of whitespace or a single repeated character.
- Remove documents with extremely low or high character count (outside reasonable bounds for a webpage or article).
- Remove or truncate documents with high repetition (e.g., the same sentence repeated over and over – these often appear in spam or error pages).
- Filter out documents with a high fraction of non-standard characters or symbols (e.g., >50% of characters are not letters or standard punctuation).
- Filter out documents containing specific disallowed content via keyword lists (profanity, extremist slogans, etc.), being mindful of context and false positives.
- Drop documents from disallowed domains or URL patterns (as discussed in URL filtering).
- Exclude documents with known boilerplate phrases or metadata (e.g., "lorem ipsum", or navigation menus if they persist).
- Language consistency: if a document's content appears to contain multiple languages or mostly a language not desired, remove or segment it.
- Remove explicit duplicates (if an exact match to another document in the set) – though this is often handled by hashing in the dedup stage.

These rules, while simple, can dramatically improve the dataset. For instance, the original Common Crawl dump contains a lot of pages that say things like "Page not found" or "403 Error" – a simple keyword check

can drop those error pages. Removing 404 pages, empty pages, and template-heavy pages can by itself eliminate a large portion of useless text. Heuristics like "if a page has more than 10% HTML tags by text, it's likely not cleaned text" can catch cases where extraction failed. Each rule may only remove a small percentage, but together they add up.

Model-Based Filtering

Model-based filtering involves using machine-learned models to evaluate the quality or content of data. These models can capture more subtle patterns than manual rules. Broadly, model-based filters fall into a few categories:

- **Language Model Perplexity Filters:** As mentioned, perplexity from a language model (LM) can indicate how *surprising* or *linguistically unusual* a text is, relative to some reference corpus. A low perplexity means the text is well-formed and similar to the training domain of the LM; a very high perplexity might mean the text is random or from a very different domain. CCNet's use of a 5-gram model per Wikipedia is one example [24]. Other pipelines have used neural LMs for perplexity – for instance, one could use a small GPT-2 model: pass each candidate document through it and measure perplexity. If perplexity > threshold, filter it out. One has to be careful in choosing the reference: a model trained on Wikipedia might consider fiction or code or slang as "high perplexity" (bad), even though those might be wanted in a diverse corpus. Thus, perplexity filtering works best when you clearly know the domain/style of text you want (e.g. if building a *formal news* dataset, a news-trained LM can flag extremely informal text). In multi-domain corpora, perplexity filters might inadvertently remove valid minority-domain data (as the CCNet authors noted [25]).

- **Trained Quality Classifiers:** Instead of a generic LM, one can train a classifier (or regression model) that directly predicts a "quality score" or "keep/drop" decision for each document. The key is having a definition of quality to train on. For example, **OpenAI's GPT-4 training** involved a filtering model: they curated a smaller high-quality dataset and a lower-quality one, and trained a classifier to distinguish them, then applied it to score all candidates. Similarly, **Datology's pipeline** (2023) trained filtering models on known high-quality corpora vs. random data [34]. The classifier might use features like embeddings or bag-of-words, or could itself be a mini language model fine-tuned to predict if a text is "good". Marion et al. (2023) and Li et al. (2024) are referenced as doing such learned filters [34]. The advantage is that a classifier can learn complex cues (like coherence, topicality, formatting) that are hard to capture with simple rules. The downside is needing labeled data for training (or proxies for quality), and the risk that the classifier's biases will shape the corpus (for instance, if it was trained on scientific articles vs. web data, it might prefer formal tone and inadvertently exclude creative language). Nonetheless, model filters have been shown to boost quality: Datology reported that model-based curation notably improved downstream model performance compared to just using the raw data [35] [36]. An example in practice: Google's *LamDA* paper described training a toxicity classifier to filter out toxic/hate content from their dialogue data. Another example: you could train a classifier to detect pages with mostly code vs. natural text, and filter accordingly.

- **Embedding-Based Similarity and Outlier Detection:** A less supervised approach is to use embeddings (vector representations) of documents to find outliers or clusters. For instance, generate a sentence embedding or TF-IDF vector for each document, then use clustering or k-nearest-neighbor to detect texts that are *very similar* (duplicates) or *very dissimilar* to the bulk of data.

Highly outlier documents might be weird content to drop; large clusters of almost identical docs might indicate a template or duplicate set – you might choose to only keep one from each cluster (this is another way to deduplicate/cluster). Techniques such as *density-based outlier detection* in embedding space can flag potentially irrelevant data. Datology's pipeline included an "embedding-based curation" step where they clustered and identified *semantic duplicates* or overly represented content [37]. This approach can catch near-duplicates that lexical hashing might miss (if phrasing is changed but meaning is same). It can also enable *targeted subset extraction*: e.g., if you have an embedding of each doc and an embedding of some target domain, you can retrieve those closest to the target (this crosses into the next category of target distribution matching).

- **Targeted Data Selection (Retrieval-based Filtering/Upsampling):** Sometimes the goal is not just to *filter out bad data*, but to *filter in the most relevant data*. For instance, if you know your model will be evaluated on biomedical text tasks, you might want more biomedical documents in your training set. Approaches like **target distribution matching** (Gururangan et al., 2020; Xie et al., 2023) use a small seed of target-domain data and retrieve similar documents from the large corpus [38]. In effect, they filter by relevance. While this is more of a *data selection* than cleaning, it's part of curation: one might intentionally filter the dataset to match a certain distribution or to include more of underrepresented categories. A practical example: if the raw data is mostly English but you want a balanced multilingual set, you'd filter (or rather partition) by language and then upsample the low-resource language portions (ensuring quality in each). Penedo et al. (2024) introduced a "dedup-informed upsampling" in their FineWeb2 pipeline, where after deduplication they *rebalance* the dataset across languages accounting for quality and duplication rates [39] [40]. This is a form of controlled curation to meet a desired composition.

It's worth noting that model-based filtering, especially if using deep models or clustering on embeddings, can be **computationally expensive** for billions of documents. Teams often combine strategies: e.g., run cheap heuristics first to cut down data size, then run a heavier model on what remains. Also, if using neural networks for filtering, one must be mindful of not accidentally introducing a bias (for instance, a model might inadvertently favor a certain style of writing). The ideal scenario is to have a well-defined notion of "quality" and possibly human-labeled examples of good vs bad, to train or validate the filter.

Case Study – Perplexity Filtering in CCNet: CCNet's authors trained 176 separate 5-gram models (one for each language supported by FastText ID). For each document, they computed perplexity per paragraph. They then examined the distribution of perplexities and intended to remove the worst third (the "tail") [41]. However, upon manual analysis, they realized some high-perplexity documents were *actually legitimate text* (just different style from Wikipedia). So they opted not to automatically drop them, acknowledging the nuance that **what's "bad" data depends on the application** [42]. In contrast, other pipelines like *Common Crawl Filtered (Wenzek et al. 2019)* did remove high-perplexity content for certain languages to focus on more standard text. This illustrates the point: model-based filters have tunable settings – one can be strict or lenient. The trend in recent projects (like RefinedWeb, LLaMA) is to lean on simpler heuristic filters and aggressive deduplication, rather than perplexity, possibly to avoid losing diversity.

Case Study – OpenAI's WebGPT Filtering: OpenAI, when preparing data for WebGPT and other models, has reportedly used a combination of a *memorization filter* (to avoid copyrighted text over a certain length), a *toxicity filter* (using Perspective API or a similar classifier to drop toxic content), and a *quality filter* (using CLIP or other signals to remove low-quality text). Although details are not fully public, this hints at a pipeline where model-based filters (some possibly pre-trained) are used to enforce content standards.

In summary, heuristic and model-based filters are complementary. **Heuristics** excel at quickly removing obvious garbage, and **models** can more flexibly identify subtler forms of undesirable content. A balanced pipeline might: (1) apply a broad heuristic sweep, (2) apply a learned filter for finer judgements, and (3) always keep a *manual review or datasheet documentation* step to record what was done and any assumptions made (since filtering can inadvertently skew the data, it's important to be transparent about it).

Scalable Deduplication: Removing Duplicates and Near-Duplicates

Duplicate data is ubiquitous in large text corpora. Websites often repeat content (think of boilerplate headers, or syndicated articles appearing on multiple news sites). Without deduplication, a dataset might contain many copies of the same text, which can lead to inefficiencies and even model overfitting or bias. Large language models have been shown to *memorize* and regurgitate training passages verbatim when those passages are duplicated many times in the training set [4]. Duplicates can also distort evaluation if the test set contains text seen during training (the model may do well simply by recall, rather than true generalization) [43] [44]. Thus, **deduplication** is a crucial part of data curation at scale.

However, deduplicating at scale is non-trivial: we need to detect duplicates or near-duplicates among billions of strings, which is computationally expensive if done naively (comparing every pair of documents is infeasible). We need efficient algorithms that approximate similarity and allow scalable detection. Two key techniques often used are **MinHash** (for Jaccard similarity of sets) and **SimHash** (for cosine similarity of feature vectors), both of which are forms of Locality-Sensitive Hashing (LSH). We will explain these and how they help catch near-duplicates.

Why Deduplication Matters

Before diving into algorithms, let's reinforce why we care about duplicates with an example. Researchers from Google and U. Pennsylvania (Lee, Ippolito, et al. 2021) found that popular NLP corpora like C4, Wiki-40B, etc., contained many duplicates [45]. In C4, they famously discovered a specific **61-word-long sentence** ("by combining fantastic ideas, interesting arrangements, and follow the current trends... in your wedding... brilliant ideas would be perfect..") that appeared **over 60,000 times** in the training set and even appeared in the validation set numerous times [5] [43]. This likely came from a spammy website with repeating text that was crawled. Because of this, a language model could easily memorize that odd sentence. Indeed, the researchers noted *over 1% of unprompted output* from a GPT-style model trained on these datasets was *verbatim memorized text* from training data [4]. By removing such duplicates, they were able to reduce the frequency of memorized regurgitations by tenfold and also eliminate train–test overlap that was affecting ~4% of validation samples [5]. In short: deduplication not only saves space, it **reduces overfitting and leakage**, making evaluations more accurate and models more general.

Another consideration is efficiency: Training on redundant data wastes compute. If 10% of your dataset is exact duplicates, you're effectively doing 10% unnecessary training steps. Deduping makes training *faster and cheaper* for the same amount of information [46] [47]. Lee et al. even found no drop in perplexity when removing duplicates – models performed **no worse (sometimes better)** on a deduplicated dataset, reaching the same accuracy in fewer steps [48] [49].

Thus, the consensus is that deduplication has *significant benefits and minimal downsides* [50]. (There has been some debate; e.g., the Pythia suite reported little effect on perplexity [51], but given the other advantages like reduced memorization, most agree it's worthwhile.)

Now, what counts as a “duplicate”? We have: - **Exact duplicates:** Two documents that are byte-for-byte identical (or identical after normalization like lowercasing and stripping punctuation). These can be removed by simple hashing (compute a hash for each document and drop those with the same hash). - **Partial duplicates / overlaps:** A document may not be wholly identical to another, but might contain a large chunk of text that appears in another (e.g., a news article syndicated across sites with different surrounding HTML). We might want to remove one of them to avoid repetition. - **Near-duplicates:** Two documents conveying the same content with minor differences (paraphrases, or copies with a few words changed, perhaps generated for SEO). These are harder to catch, but might still be worth removing to maximize diversity of content in the dataset.

The strategies differ for exact vs. near duplicates. Let’s break down approaches:

Exact Deduplication

For exact duplicates, the approach is straightforward and scalable: compute a cryptographic hash (MD5, SHA-1, etc.) of each document’s text. Two identical texts will have the same hash. You can then use a set or database index to find collisions (i.e., if a hash is seen again, mark that document as duplicate). This is **O(N)** in the number of documents (computing the hash for each is linear in total data size), which is very feasible. The memory can be an issue if billions of hashes must be stored, but using a truncated hash (e.g., 64-bit) can mitigate that. CCNet did exactly this at paragraph level with 64-bit SHA1 hashes [52] [53]. Storing billions of 64-bit hashes is manageable in a distributed setting. One must also consider accidental collisions (two distinct texts having same hash) – with 64-bit, collisions are possible but extremely rare, and acceptable for this use (a tiny risk of mistakenly dropping a non-duplicate).

One complication is that if you want to deduplicate *across shards* or *crawls*, you need a global method. It might be easier to deduplicate within each month’s crawl (to reduce memory pressure), but then duplicates across months could slip through. Some pipelines (like CCNet) mention the option of cross-shard vs. within-shard dedup [54] [55]. **RefinedWeb** addressed Common Crawl’s immense scale by deduplicating within each crawl dump first, then possibly on combined sets; they also employed fuzzy matching (explained next) to go further [29].

Exact deduplication is lossless in the sense it only removes truly identical items. It’s a necessary first pass but not sufficient because many near-duplicates will remain.

Near-Duplicate Detection with MinHash (Jaccard Similarity)

MinHash is a technique to efficiently estimate the similarity of two sets. When we talk about document similarity, a common measure is the **Jaccard similarity**: for two sets *A* and *B*, Jaccard = $|A \cap B| / |A \cup B|$. In text, we often represent a document as a set of “shingles” (substrings of length k, or sometimes words) and use Jaccard to measure overlap. High Jaccard means the documents share a lot of shingles (i.e., they have a lot of text in common). For example, if we use word-level shingles, two copies of the same article (even with a few minor edits) will have Jaccard similarity very close to 1, whereas completely different documents have Jaccard 0. We want to catch cases with similarity above some threshold.

The problem is computing Jaccard for all pairs of documents is extremely expensive. If you have *N* documents, comparing all pairs naively is O(N^2). MinHash comes to the rescue by creating a much smaller

signature for each document such that the similarity of two signatures approximates the Jaccard of the full sets [56] [57].

How MinHash Works (Conceptual): The idea is brilliant in its simplicity. Imagine each document is represented by a set of tokens (say, all its 5-word shingles). If we list all unique tokens that appear in any document as the "universal set" and assign each a unique index, we could represent each doc as a boolean vector (index = 1 if token present, 0 if not). MinHash involves **randomly permuting** the rows of this conceptual matrix and then recording, for each document (each column), the first token index (row) that appears in that document under that random permutation [58] [59]. That index is the *minhash* value for that document under this permutation. If two documents share a lot of tokens, there's a higher chance that their first token under the permutation is the same (because they likely share the "min" token). In fact, one can prove: *the probability that two docs have the same minhash value (for a random permutation) is exactly equal to their Jaccard similarity* [60] [57]. This is a key property [61]. Therefore, if you generate many independent random minhash values (say k different random permutations), and compare the signatures (a vector of k minhash values) for two documents, the fraction of signature entries that match is an unbiased estimator of the Jaccard similarity [62] [63].

In practice, we don't explicitly permute huge vectors. Instead, we simulate it using hash functions. We choose *k* different hash functions that map tokens to integers (these serve as "random orderings" effectively). For each document, for each hash function, compute the hash of each token in the doc and take the minimum hash value. That yields *k* numbers = the MinHash signature for that doc [64] [65]. With a properly chosen hash family, those mimic random permutations.

Now, comparing all signatures pairwise is still heavy if *N* is large, but since each signature is just k numbers, one can use **LSH bucketing**: e.g., hash parts of the signature to create buckets of candidate similar docs. Typically, you split the signature into *b* bands and hash each band; if two docs share a band exactly, they are candidate near-dups. This uses the locality-sensitive hashing principle to drastically reduce comparisons. The result is you don't compare all pairs, only those that hash into the same bucket for at least one band (which is likely only if they share a lot of minhash entries).

MinHash Example: Suppose we have two short documents: - Doc1: "the cat eats fish" - Doc2: "a dog eats fish"

Let's use word-level shingles (here the set of unique words in Doc1 is {the, cat, eats, fish}, in Doc2 is {a, dog, eats, fish}). The Jaccard similarity = |intersection| / |union|. Intersection is {eats, fish} (they both contain those words), size 2. Union is {the, cat, eats, fish, a, dog}, size 6. So Jaccard = 2/6 ≈ 0.333. Now, to apply MinHash, we pick some hash functions. For simplicity, assign each word an ID: a=1, cat=2, dog=3, eats=4, fish=5, the=6 (just an example mapping). A random hash function (permutation) might map these IDs to new values. Say one MinHash function H1 is defined (randomly) as: H1(x) = (2*x + 1) mod 7 (just a toy example). Compute H1 for each token in doc: - Doc1 tokens and H1 values: {the(6)→ (2*6+1 mod7)=? (2*6+1=13 mod7=6), cat(2)→ (2*2+1=5 mod7=5), eats(4)→ (2*4+1=9 mod7=2), fish(5)→ (2*5+1=11 mod7=4)}. The minimum H1 value among Doc1's tokens = 2 (from "eats"). - Doc2 tokens and H1: {a(1)→3 mod7=3, dog(3)→(7) mod7=0, eats(4)→2 mod7=2, fish(5)→4 mod7=4}. Minimum = 0 (from "dog").

So under H1, MinHash(Doc1)=2, MinHash(Doc2)=0, not equal in this round. If we had another hash function H2 (another random permutation), we'd do similarly and get another pair of mins. With enough different

hashes, the fraction of times MinHash(doc1)==MinHash(doc2) would approach 0.333 (the true Jaccard). If we used, say, 100 hash functions, we might expect around ~33 of them to produce collisions for doc1 and doc2.

MinHash signatures can thus be compared: if two docs have identical signatures (all k values same), they are almost certainly very similar. Usually we don't require perfect match; instead, LSH will group those with many matching entries.

Scaling with MinHash: The beauty is that each document's signature is small (say k=128 integers). We can store that and toss the full text. To find duplicates, use an LSH scheme to get candidate pairs and then optionally verify their actual Jaccard using the signature (or even re-read the text for a final check if needed). This turns an O(N^2) problem into something like O(N * k) for computing signatures plus the LSH lookup overhead which is sub-quadratic. It's how companies like Google tackled near-duplicate detection across billions of webpages back in the day [66] [67].

In fact, **Broder (1997)** [68] is the seminal paper introducing shingling and MinHash for finding near-duplicate web pages. Our discussion is essentially based on that approach. It remains highly relevant; e.g., the Lee et al. (2021) team used a hash-based method akin to Broder's to identify document pairs with high n-gram overlap [69].

SimHash (Locality-Sensitive Hash for Cosine Similarity)

SimHash is another technique, famously used by Google (Charikar 2002), to detect near-duplicates with high efficiency. While MinHash targets Jaccard similarity (good for set overlap), **SimHash targets cosine similarity** of feature vectors. In practice, SimHash is often applied on a representation of the document (like a bag-of-words TF-IDF vector) and produces a single *fingerprint*, e.g., a 64-bit hash. The neat property is that *similar documents will have similar fingerprints*, in the sense that their 64-bit hashes differ in only a few bits (Hamming distance). So to find near duplicates, one can look for hashes that are within a small Hamming distance threshold.

How SimHash Works (conceptual): Imagine a document represented by features (like words with weights). SimHash randomly projects this high-dimensional vector to a lower dimension bitstring. One way to do this: initialize an array of, say, 64 accumulators with 0. For each feature (word) in the doc, take its hash to generate a 64-bit random vector; then add or subtract that vector from the accumulators, weighted by the feature's weight (e.g., TF-IDF). After processing all features, each accumulator position will have a positive or negative sum. Finally, set the i-th bit of the SimHash to 1 if the i-th accumulator is positive, or 0 if negative [70]. This way, each bit captures a random projection of the vector. Two documents with similar content will tend to push the accumulators in the same direction for many bits, yielding similar bit patterns. The *Hamming distance* (number of bit positions that differ) between two SimHash outputs correlates with the angle between the original vectors.

Google used SimHash to detect duplicates in their web index, as it's extremely fast (just compute one 64-bit number per doc) and storage-light [66] [71]. Finding near matches means finding other hashes within a few bits difference – again can be done with clever indexing (e.g., treat each hash as a point in 64-d Hamming space and use LSH or brute force within small radius since there are techniques to search Hamming balls).

MinHash vs SimHash: - MinHash is more directly interpretable (approximate Jaccard) and flexible (choose shingle size, etc.). But generating a large signature per doc and LSHing it can be heavier than SimHash's

single fingerprint. - SimHash is very lightweight, but you have to set a Hamming threshold which indirectly corresponds to some similarity. Some research found SimHash finds more near-dup pairs than MinHash in certain settings, and can have better precision in web page de-duping [71]. Many modern pipelines mention using MinHash explicitly (likely because it's easier to tune for a known Jaccard threshold). - In practice, either or both can be used. For instance, a pipeline might first do exact hash dedup, then SimHash for very near duplicates, and then MinHash for broader near-dup detection. Or choose one scheme depending on needs.

RefinedWeb's Dedup Strategy: According to descriptions, RefinedWeb (Falcon's dataset) did a multi-step dedup: First, remove *exact duplicates* (document-level hashing, done per shard because CC data is huge) [72]. Next, remove overly repetitive content (docs with lots of repeated lines) *before* fuzzy dedup, to avoid false positives [29]. Then, they used a **MinHash-based pipeline** to catch near duplicates, which they noted was *"very effective at removing SEO templates"* – those are pages that aren't exact duplicates, but share a common template text with minor differences across many sites [29]. They used quite strict settings, resulting in a much higher removal rate than prior datasets, meaning they aggressively tossed out anything that looked like a duplicate cluster, presumably to maximize unique content [20]. This cleaned corpus was significantly smaller but arguably higher quality and unique.

Another dedup aspect is **intra-document duplicate removal**: sometimes a single document might have large repeated segments (e.g., some forums or article pages accidentally include the same paragraph twice). Deduping at paragraph level (like CCNet did) inherently removes those as well by dropping duplicate paragraphs [19]. So dedup can also improve the quality of single documents by eliminating repeated boilerplate.

When Not to Deduplicate: An interesting edge: if your data includes, say, dialogues or repetitive formats (like question-answers that legitimately repeat structure), dedup might remove something you wanted. But in general, with web text, duplication is rarely intentional content. The one case to be cautious: if building a dataset for language modeling, you might *want* multiple occurrences of some popular texts (like famous quotes) to reinforce them. But usually, it's better to have unique instances and let frequency reflect actual frequency in natural data, not artificial duplication from crawling.

To sum up this section, here's a quick guide to **implementing dedup at scale**: 1. **Normalize** documents in a simple way (lowercase and remove punctuation maybe) to avoid trivial mismatches. 2. **Exact hash deduplication:** Compute a hash for each doc, drop duplicates. (Also consider doing this for paragraphs or n-grams to remove embedded duplicates.) 3. **Near-duplicate detection:** - Option A: Compute MinHash signatures for each doc. Use LSH to group candidates and remove duplicates above a Jaccard similarity threshold (e.g., >0.8 similarity). - Option B: Compute SimHash for each doc. Use a fast approximate neighbor search to find docs within e.g. 3-bit Hamming distance, and merge those groups. - Option C: Use both: perhaps SimHash to quickly catch very close duplicates, and MinHash for catching those with substantial overlaps. 4. **Be mindful of memory and speed:** Use efficient data structures, maybe distributed computing (MapReduce style or Spark) if dealing with TBs of data. 5. **Verification:** For critical cases, one might verify duplicates by actually comparing texts if needed to avoid any false positive from the hashes (though usually if the similarity metric is set high enough, it's fine). 6. **Remove duplicates from validation/test as well relative to training:** i.e., ensure no document in validation is present in training (to ensure fair evaluation). This is a form of dedup across splits.

When done, the dataset will have each piece of text appearing only once (to the best of the algorithm's ability). This greatly **increases the effective diversity** of the data per token count, and addresses issues of memorization.

Finally, it's good to log how much data was removed during dedup. For example, "We found 1,234,567 duplicate pairs and removed 5% of documents as duplicates." Documenting this in the datasheet is valuable because it tells users how much redundancy was present in the raw data versus the final data.

Case Studies and Examples

To concretize the above concepts, let's look at a few real-world examples of large-scale data curation efforts. These case studies illustrate the pipeline choices, challenges, and outcomes from filtering and deduplication.

Case Study 1: C4 (Colossal Cleaned Common Crawl) – A Web-Crawl Pipeline for T5

C4 is a ~750GB English text dataset derived from scraping the public Common Crawl, introduced by Raffel et al. (2020) for training the T5 transformer model. The process to create C4 was one of the first widely discussed attempts at cleaning Common Crawl at scale: - **Data Source:** Multiple snapshots of Common Crawl's WET files (plain text extractions). - **Extraction & Cleanup:** They used an existing boilerplate removal algorithm to get the main text of each page. They then applied heuristics: remove non-English pages (using a language classifier), remove pages with obscene or problematic content via a blocklist (which, as noted, accidentally filtered some technical pages like those containing "JavaScript" due to an overly broad pattern) [31]. - **Quality Filtering:** They dropped pages that were near-duplicates of Wikipedia (to avoid copy-pasted Wikipedia content, since that can be obtained separately). They also removed short pages and some known problematic domains. - **Deduplication:** The C4 paper doesn't detail all dedup steps, but it's known they did some amount of dedup (likely exact-match dedup of lines or paragraphs). Nonetheless, as Lee et al. (2021) found, C4 *still* had duplicates, including the infamous 61-word repeated text [43]. So whatever dedup they did wasn't exhaustive (they perhaps focused on exact matches and missed near-dups). - **Result:** A huge text corpus that is relatively cleaner than raw Common Crawl, but not perfect. It became a standard benchmark dataset. The makers acknowledge trade-offs in filtering: for example, blocklists removed some content unintentionally (which was later fixed in a revised version called C4 V2).

Lesson: Simple pipelines (language filter + badword filter + basic dedup) can produce a workable dataset, but careful tuning is needed to avoid over-filtering or under-filtering. The C4 experience showed the importance of reviewing filtering rules (e.g., the "JavaScript" issue). It also showed dedup is hard – what looks clean still had a sizable percentage of duplicates until more sophisticated methods were applied later [5].

Case Study 2: Deduplicating C4 and Other Datasets (Lee et al., 2022)

We've referenced this work multiple times – it systematically examined the effect of deduplication: - They took C4, Wiki-40B, RealNews, and another corpus (One Billion Words) and applied a new dedup pipeline with both exact substring matching and approximate whole-document matching [69]. - Tools: One tool found any substring longer than 50 characters that appears in multiple places (to catch partial duplicates like a common footer text across pages). Another tool was essentially the MinHash approach for whole-document comparison [69]. - They deduplicated C4 thoroughly, removing a lot of content. Their metrics

showed no performance loss in the trained models, and they actually got slight improvements in some cases [50]. - An important outcome was highlighting *train-test contamination*: They demonstrated a concrete example where a chunk of text was in both train and validation (the wedding example) [43], and how such overlap could inflate evaluation scores. After dedup, some language model's perplexity on validation increased slightly (which is *good*, because the overfitting was removed, giving a truer measure of generalization). - They released their code and the message was clear: **dedup your data!** This case study is a strong evidence-based endorsement for deduplication as a best practice.

Case Study 3: The Pile (2020) – Curated Diverse Sources vs. Web

The Pile is an 800GB open dataset from EleutherAI, built by concatenating 22 different data sources (including academic journals, books, Wikipedia, StackExchange, GitHub, and some Common Crawl-based subsets like OpenWebText2). The Pile's approach to quality was to select **high-quality sources** upfront, rather than filtering a raw crawl. For example, they included all of Project Gutenberg (books), all of ArXiv, etc., which are presumably high-quality by nature. For the web crawl portion, they didn't take random pages; they took *OpenWebText2*, a recreation of the WebText (used in GPT-2) which itself is web pages linked on Reddit with score > 3 (i.e., somewhat curated by human interest). This approach yields a very diverse set and avoids a lot of garbage by design (e.g., Reddit links often point to news or blogs rather than spam). - They still performed deduplication – in fact, deduping *across sources* too (to ensure no document from one source overlaps with another). E.g., some Wikipedia text might appear in ArXiv references etc., so they checked for overlaps. - The Pile didn't heavily filter for offensive content (some sub-corpora have their own filters, but generally they aimed to preserve data "as is" given the sources were mostly already moderated). - Outcome: The Pile proved a very strong training set for language models, often outperforming models trained on straight Common Crawl, suggesting that *curation by source* can be effective. However, assembling such sources requires domain knowledge and often permission (most are public though). It's a different philosophy from scraping "everything" then filtering.

Comparing The Pile to RefinedWeb is interesting: **RefinedWeb (Falcon)** chose to stick purely with Common Crawl but filter it so well that the end result rivaled or beat The Pile's quality [7]. Penedo et al. (2023) showed that *"properly filtered and deduplicated web data alone can lead to powerful models, even significantly outperforming models trained on The Pile"* [7]. They extracted on the order of **5 trillion tokens** of raw web text and after aggressive filtering distilled it to a high-grade 500 billion token dataset (which they released publicly) [73]. Falcon LLM trained on this RefinedWeb corpus achieved great results, supporting their claim that you don't necessarily need curated sources if you have enough web data and a good pipeline. This is a testament to data curation: *quantity + curation can substitute for manual quality selection*. It also alleviates concerns of running out of high-quality text (since curated sources like books or Wikipedia are limited, but web is vast if you can purify it) [7].

Case Study 4: FineWeb2 (2024) – Multilingual Curation at Scale

The **FineWeb/FineWeb2** project by Hugging Face (Penedo et al., 2024) extends these ideas to *1000+ languages*. Scaling to diverse languages is challenging because quality signals (like certain heuristics or available models) differ per language: - They built an adaptive pipeline where language identification is first (to route text to the appropriate language-specific filters) [23]. For each language, they customized filtering thresholds (since, e.g., what constitutes too short or which characters are common differs). - They also faced the lack of pre-trained models for some low-resource languages, so they relied more on heuristic filtering for those. - Deduplication is even more crucial in multilingual data because certain languages (like

boilerplate in English) could pollute others – e.g., an Arabic page might have an English boilerplate that gets deduped out for consistency. - FineWeb2 pipeline comprised steps: LID, Dedup, Filtering, then a novel step called "Rehydration" which upsamples data from languages that lost a lot due to dedup (to ensure each language has sufficient volume, taking into account both quality and quantity) [39]. - They evaluated on tasks in several languages to ensure the pipeline decisions (like thresholds) actually improved downstream performance, doing extensive ablation. Ultimately, FineWeb2 produced a **20TB text dataset with 5 billion documents in 1000 languages** [74] – likely the largest multilingual corpus of its kind, curated automatically. - A key result: models trained on FineWeb2's data outperformed those trained on prior multilingual sets, demonstrating the importance of careful curation even beyond English [75] [76].

Lesson: Multi-language curation requires careful per-language handling. Also, the concept of *"dedup-informed upsampling"* (don't upsample stuff that is itself duplicated content, and after dedup, consider boosting languages that were mostly duplicates of English content for example) is an interesting innovation [40] [39].

Case Study 5: SlimPajama (2023) – Cleaning a Public Dataset

SlimPajama is a project by Cerebras that took the RedPajama dataset (which was an open reproduction of the LLaMA training data, ~1.2 trillion tokens from various sources) and cleaned/deduplicated it extensively to make a smaller, high-quality dataset of 627B tokens [77] [78]. They removed duplicates and filtered out low-quality pieces. This is a practical example of "filter a public dump & generate a datasheet" – RedPajama was the public dump (a mix of CommonCrawl and others) and SlimPajama is the refined version: - They used dedup at multiple levels (exact and fuzzy). - They filtered toxic content and non-English (since LLaMA was mostly English). - They ended up removing about half the tokens while actually improving model performance, indicating a lot of the original data was indeed not needed or harmful. - They documented the process in a technical report, effectively creating a datasheet-like description of SlimPajama.

SlimPajama highlights that *even already curated collections can benefit from further cleaning*. Many "open" datasets incorporate CommonCrawl content that might have been only lightly filtered; there's always room to apply newer techniques (like better neural filters or updated blocklists) to improve them.

These case studies reinforce the core principles: - There is a trade-off between **keeping more data** (to have more variety) and **keeping only high-quality data**. Different projects pick different points on this spectrum. But it's increasingly evident that *more data isn't better if it's full of duplicates or junk*. You get diminishing returns or even negative effects (like overfitting or model confusion). - Deduplication is universally beneficial across these examples. - Modern pipelines like RefinedWeb and FineWeb show that with heavy filtering, you can use the raw web to get results that rival hand-curated sets, which is promising for scalability. - Each pipeline needs to be tailored: what works for English might not directly work for Chinese or Arabic; what works for scientific text might differ from casual text. - **Documentation** (our next topic) is crucial, because one can only interpret a model's performance or potential biases if we know what went into the training data. For example, knowing that "We filtered out all Reddit comments from our data" or "We heavily upsampled fiction books" could explain certain model behaviors. Without a datasheet or similar, users of the data (or models trained on it) are left guessing.

Documenting the Dataset: Datasheets and Data Statements

After performing data curation – applying all those filters and dedup rules – we end up with a dataset that is hopefully clean and suitable for training. The final, and very important, step is to **document what we have done and what the dataset contains**. Good documentation helps downstream users (researchers, engineers, or even auditors) understand the strengths and limitations of the dataset. It also promotes transparency and accountability: others can scrutinize or replicate the process if it's well documented.

Two influential concepts in dataset documentation are **"Datasheets for Datasets"** (Gebru et al., 2018) and **"Data Statements for NLP"** (Bender & Friedman, 2018). Both emerged from the recognition that machine learning datasets, like electronic components or medical data, should have standardized documentation about their creation and characteristics to prevent misuse and to flag biases [79] [2].

Datasheets for Datasets (Gebru et al., 2018)

Gebru and colleagues propose that every dataset release be accompanied by a *datasheet*, analogous to a product datasheet in electronics [80]. The datasheet would include information such as: - **Motivation:** Why was this dataset created? What specific needs or questions is it addressing? (E.g., "This dataset was compiled to train large English language models for general-purpose NLP research.") - **Composition:** What is in the dataset? Details on data sources (e.g., "text scraped from Common Crawl, spanning 2016-2023, primarily English"), the total size (documents, tokens), the breakdown by categories (languages, domains, etc.), and any preprocessing done (like "removed HTML, lowercased text"). - **Collection Process:** How was the data collected and filtered? Here we describe the pipeline steps – which we've done extensively in this guide. For instance, outline that "raw web data was downloaded from CommonCrawl, then we applied language detection using FastText, removed non-English texts, filtered out documents with offensive content using a blocklist and a toxicity classifier, and deduplicated the remaining text using MinHash (threshold X)." This section should be very explicit so that someone else could in principle reproduce a similar dataset or at least understand its provenance [81] [82]. - **Preprocessing and Cleaning:** (Some overlap with collection process.) List any transformations on the data: tokenization, normalization, any labels added, etc. E.g., "Removed all XML/HTML tags, normalized unicode, split text into sentences..." or "Converted all text to lowercase and removed punctuation, except within words." This matters especially if these choices could affect usability (like if someone expects the original casing, they should know it's gone). - **Uses:** What is this dataset intended for *and* not intended for? For example, "Recommended use: pre-training general language models or as a benchmark for text compression. Not recommended for: tasks requiring knowledge of texts after 2023 (since crawl is up to 2023), or for studying conversational data (since it mostly contains written text, not dialogue)." Also any known tasks it's been used in successfully. - **Distribution & Maintenance:** How is it distributed (e.g., on HuggingFace or via torrents)? License information (very important if it contains copyrighted web text – often these curated sets are released under a certain assumption of fair use or an open license if possible). Who is maintaining it or will there be updates? Any versioning (like "This is version 1.0, created August 2025, using CommonCrawl up to July 2025"). - **Ethical Considerations & Privacy:** Does the dataset have any sensitive content? Did we filter personal data (emails, phone numbers)? If not, warn that it might contain some personal info because web data can. Are there biases present? For example, if much of the web data is from the US, the dataset might under-represent other cultures or dialects. Document known biases: "Likely over-represents English news articles and under-represents conversational text or low-resource languages." Or "We observed that our filtering might have disproportionately removed text written in African American Vernacular English due to perplexity thresholding, which could bias models against that dialect." These sorts of disclosures are crucial for

responsible use [79] [6] . - **Limitations:** What the dataset doesn't cover. Maybe it lacks data from certain time periods (no texts earlier than year X, or nothing from social media if those were filtered out). Or it might be skewed towards certain topics (maybe because of how CommonCrawl samples the web). - **Creator/Curator Info:** Who made the dataset? (People or organizations, to contact for questions.) Also, if different from the original data source authors (like CommonCrawl did the crawling, we did the curation – we should credit CommonCrawl and any other source). - **Legal/Licensing:** E.g., "Contains data from public websites; licensing status of individual texts is unknown, use at your own risk in commercial settings" or if care was taken to include only certain licensed content, say so. - **Acknowledgements:** If this work builds on others (like using someone's code or pipeline), or was funded by something, mention it.

Gebru et al. provide a list of questions to guide filling out such a datasheet [82] [83] . The idea is not that every datasheet is identical, but that by answering those questions, dataset creators are forced to think about issues that otherwise might be overlooked (like potential harms, biases, etc.).

Importantly, datasheets are intended to be done *conscientiously*, not auto-generated [84] . The process of writing one can reveal issues (maybe you realize you don't actually know if some offensive content remains because you didn't check – that might prompt you to do an extra check).

In our context, after filtering a public web dump, a datasheet would detail all the filtering steps (so others know exactly what was removed) and reasons for them, plus info about the resulting data.

Data Statements for NLP (Bender & Friedman, 2018)

Bender & Friedman introduced "data statements," which are similar in spirit to datasheets but tailored to language data and with a particular emphasis on documenting **sociolinguistic aspects** to mitigate bias [6] . They were concerned with cases where a dataset might be language from one population but used to develop tech for another, leading to bias (e.g., speech recognition trained on primarily White American voices might do poorly on other accents – a data statement should expose the demographic makeup of the data) [85] .

A data statement for an NLP dataset would include things like: - **Curation Rationale:** Why these texts? (Similar to motivation). - **Language Variety:** What language(s) or dialects are present? If English, is it predominantly American English? Any info on that. If multiple, list them. - **Speaker Demographics:** For datasets of spoken or written content where authors are known, who are the speakers/authors in terms of demographics (age, gender, ethnicity, etc.) if known. For web data, this is usually unknown, but one might infer some (e.g., content is mostly from US websites so implicitly mostly US English authors). - **Annotator Demographics:** If the data has annotations (not our case, since it's just raw text), document who the annotators are, as their perspectives might influence labels. - **Content and Genre:** What kind of language is it? (Conversation transcripts, news articles, tweets, etc.) - **Recording or Collection environment:** If applicable (like for audio). - **Provenance:** Where exactly did the data come from (which websites, etc., any selection bias like only high-ranked pages on Google? In our case CommonCrawl tries to be broad, but note if any filtering by domain). - **Known Biases:** E.g., "The corpus likely under-samples non-Western perspectives because it's largely English web text and English web content skews Western-centric. It may over-represent technical content because of including a lot of StackExchange and ArXiv, which could bias a model towards those domains." - **Miscellaneous:** Anything else about distribution (if texts are predominantly formal, etc.), and how missing data was handled (not relevant for raw text).

The goal is similar to datasheets: provide **context** so that a model trained on the data isn't a black box in terms of data origin. It helps users make better decisions. For example, a data statement might reveal "speakers in this dataset are 70% male" – so if someone uses the model and finds it works less well for female topics, they can correlate with data imbalance.

In our scenario, since we're dealing with web text, we likely cannot detail author demographics, but we can mention proxies: e.g., majority of content likely from the USA and Europe (if many .com, .uk, .de domains were present), which might reflect primarily those cultural contexts. And we can mention if we included content from social media or not – because that influences style and bias (social media language vs. formal).

Putting It Together: Creating a Datasheet for a Filtered Web Dump

Let's imagine we filtered a public Common Crawl dump (English pages) with steps as described (badword removal, dedup, etc.) and now we want to produce a *datasheet* for our new dataset "CleanWeb-1B" (say it has 1 billion tokens of English text from 2025). What would we include? A brief outline:

- **Motivation:** "This dataset was created to support training of large-scale English NLP models, addressing the need for a high-quality web-derived corpus with minimal noise and reduced biases. It aims to be a benchmark for web-scale data quality."
- **Composition:** e.g., "Contains ~20 million documents, ~1 billion tokens. All documents are English plaintext web pages from CommonCrawl snapshots (Jan–Jun 2025). The dataset primarily includes news articles, blog posts, forums, and other text-rich web pages. We removed pages that were mostly code or data tables. The final content is 55% news/blog, 20% forum discussions, 15% technical documentation, 10% others (approximate)." – If we have such stats from analysis.
- **Collection Process:** "Data was collected from CommonCrawl. We used the WET files (plaintext extractions). We filtered out documents via the following pipeline – (1) Removed documents from known spam or adult domains using domain blocklists (4.8 million domains blocked). (2) Performed language identification using FastText; non-English or low-confidence documents were discarded (~5% of data). (3) Applied heuristic content rules: removed docs < 100 characters or with >50% non-alphanumeric characters, etc. (4) Applied a toxicity classifier (open-source model) to remove very toxic content (removed ~0.1% of documents flagged for hate speech or extreme profanity). (5) Deduplicated the remaining documents: exact deduplication via hashing removed 1.2% of docs; near-duplicate detection via MinHash removed an additional 3.5% of docs, to drop documents with Jaccard similarity >0.8 to any other. (6) Manual spot-checking of a random sample was done to verify quality. We then shuffled and split 99% train, 1% validation." This thorough description aligns with what we've learned and is exactly the kind of info a datasheet needs [81] [82].
- **Recommended Uses:** "This dataset is intended for unsupervised pre-training of language models. It could also be used for research into data quality filtering. It is not recommended to use this dataset for studying conversational language or social media dialects, as those are not well-represented. Not suitable for applications needing real-time data (as all content is pre-2025)."
- **Distribution:** "Available on Hugging Face datasets hub under CC-BY license (for the curation, though original underlying content may have various copyrights). Dataset is 10GB compressed. No PII was intentionally included, but some personal names may exist in news articles – we advise caution if using in production."
- **Known Issues:** "Despite filtering, some offensive or biased content may remain, as our filters are not perfect. The dataset may underrepresent languages other than English (non-English were removed) and varieties of English outside standard writing. There is likely an overrepresentation of US-centric

content and tech topics due to CommonCrawl's distribution. Also, because we removed toxic language, the dataset is cleaner than the raw web; models trained on it might be less familiar with slurs or harmful language, which could be a pro or con depending on use."

- **Ethical Considerations:** "We acknowledge the potential biases: e.g., mainstream web content often underrepresents marginalized voices. We attempted to mitigate blatant hate speech via filtering, but subtle biases (gender stereotypes, etc.) in the source data are not wholly removed. Users should be aware that models trained on this data may still exhibit biases present in online text. We encourage testing models for biases and fairness."
- **Caveats:** "One should not assume this dataset covers all topics equally. Medical and legal information in it is not verified for accuracy. Any use in a deployed system should consider the possibility of incorrect or outdated information in the data."

This hypothetical datasheet illustrates how we communicate what we did and what's inside. It combines elements from both datasheets and data statements: the process and technical aspects, and the ethical/social aspects like bias.

It's also good to mention if any part of the pipeline had a significant human oversight (like "20 pages were manually examined to tune the toxicity threshold").

By providing such documentation, we **enable better science** and responsible AI development [86]. As Bender & Friedman argue, without these data statements, NLP practitioners might unknowingly train models on data from one demographic and deploy on another, leading to poor performance or bias [85]. A data statement/datasheet helps make those details explicit so that, for example, a practitioner sees "ah, this corpus is mostly formal written text – maybe I shouldn't expect it to do well in chat conversations" or "this dataset filtered out all profanity – if I deploy a chatbot, it might not know how to respond to insults appropriately."

In recent years, there's been an increasing expectation in the ML community that major datasets come with such documentation. Conferences and journals encourage or even mandate it. For instance, the BigScience project delivered a very detailed datasheet for the BLOOM model's training data (ROOTS corpus), and many HuggingFace datasets have "Dataset Cards" which are like mini-datasheets.

Conclusion: In the big picture, data curation at scale isn't just about cleaning data – it's about doing so **systematically** and **transparently**. Web-scale datasets feed into models that can impact society, so ensuring quality (via filtering and deduplication) and openly documenting the data (via datasheets or data statements) are steps toward more reliable and ethical AI.

Summary & Key Takeaways

To wrap up, let's summarize the major points covered about data curation at scale:

- **Scaling Challenges:** Large-scale datasets (billions of words) can't be curated manually; automated pipelines are needed. These pipelines often involve multiple stages to incrementally refine raw data into a high-quality dataset [10]. Key challenges include handling noise, variety of content, and computational efficiency.

- **Quality Filtering:** Use a combination of **heuristic filters** (length, character checks, blacklist/whitelist rules, etc.) and **model-based filters** (language model perplexity, trained classifiers for quality or toxicity) to weed out undesirable content. Heuristics are fast and interpretable, while models can capture complex patterns [87] [34]. It's important to tune these filters to not remove useful data inadvertently; often an iterative process with evaluation on sample data is used to set thresholds.

- **Deduplication:** Duplicates are pervasive in web data (e.g., boilerplate or re-posted content) and can lead to model memorization and evaluation leakage [43] [4]. Deduplication is therefore essential. **Exact deduplication** (hashing) removes identical texts cheaply. **Near-duplicate detection** via algorithms like **MinHash** (estimating Jaccard overlap) and **SimHash** (fingerprinting content) allows catching non-exact duplicates efficiently [69] [29]. Scalable LSH techniques enable deduping billions of items. The end result is a dataset with significantly reduced redundancy – beneficial for model training speed and generalization [5].

- **Case Studies:** Real-world pipelines (C4, CCNet/LLaMA, The Pile, RefinedWeb, FineWeb, etc.) illustrate different approaches. For example, C4 applied basic filtering and missed some duplicates, whereas RefinedWeb applied heavy filtering and aggressive dedup to great effect [7]. The Pile curated by source vs. RefinedWeb curated by filtering a giant crawl shows there are multiple paths to a high-quality dataset. Recent efforts (SlimPajama, FineWeb2) indicate the community is continuously improving data curation methods (deduplicating more thoroughly, filtering more intelligently, addressing multilingual issues, etc.).

- **Documentation:** After curation, producing a **datasheet** or **data statement** is critical. This documentation should describe the dataset's motivation, composition, collection process (including all filters and heuristics used), intended uses, and any ethical considerations [2] [6]. Transparency about what was removed (and why) helps others understand potential biases or gaps. For instance, if you removed all profanity, note that so users know the dataset (and any model from it) might under-represent toxic language (which can be a feature or a bug depending on application). Documentation frameworks like **Datasheets for Datasets** [2] and **Data Statements** [6] provide templates for covering relevant details. Ultimately, a well-documented dataset fosters trust and reuse in the research community.

- **Impact on ML Models:** Properly curated data leads to **better models**. By removing noise and duplicates, models train faster and often perform better on eval sets (since they aren't overfitting junk or seen data) [5]. Conversely, poor curation can lead to inflated expectations (a model that appears great on validation might just be echoing training data) or problematic outputs (a model spouting memorized private info from training data, which dedup could have removed). Curation and documentation together also contribute to the **ethical development** of AI – e.g., by filtering extreme hate content, we reduce the chance a model generates it; by documenting demographics, we highlight if a model might be biased.

To any graduate student or practitioner stepping into large-scale dataset creation: **treat the data pipeline as a first-class research problem**. The field has evolved from assuming "more data is always better" to recognizing "better data is better." Techniques like MinHash and careful filtering are powerful tools in your arsenal to achieve that better data. And just as you would meticulously document an experimental procedure in a lab, do so for your dataset. This ensures that others (and your future self) can understand and build upon your work responsibly.

In summary, data curation at scale is a multifaceted process: it's part engineering (handling huge volumes), part linguistics (understanding content and context), part ethics (deciding what to remove or keep), and part documentation (communicating what was done). Mastering these aspects will enable you to create high-quality datasets that drive the next generation of machine learning models, while also aligning with principles of transparency and fairness in AI.

References (Key Literature):

- Gebru, T., et al. (2018). *"Datasheets for Datasets."* [2] [88] Introduces the concept of datasheets to document ML datasets, improving transparency and accountability.

- Bender, E., & Friedman, B. (2018). *"Data Statements for NLP."* [6] Proposes a template for detailing the characteristics of language data to address bias and ethical issues.

- Lee, K., et al. (2022). *"Deduplicating Training Data Makes Language Models Better."* [4] [43] Empirical study showing benefits of removing duplicates in text corpora (less memorization, fairer evaluation).

- Penedo, G., et al. (2023). *"The RefinedWeb Dataset for Falcon LLM."* [7] [73] Demonstrates that scaling up web data with strong filtering/dedup can produce state-of-the-art models without relying on curated sources.

- Penedo, G., et al. (2024). *"FineWeb2: One Pipeline to Scale Them All."* [75] [8] Describes a multilingual data curation pipeline that adapts filtering and deduplication to 1000+ languages, releasing a massive high-quality dataset.

These works, among others [69] [29] [10], form the foundation of modern data curation practices. As you engage with creating or using large datasets, keep these principles and examples in mind – they will help ensure your data (and models trained on it) are as robust, reliable, and unbiased as possible.

---

[1] [2] [3] [33] [79] [80] [81] [82] [83] [84] [88] [1803.09010] Datasheets for Datasets
https://ar5iv.labs.arxiv.org/html/1803.09010

[4] [5] [49] [2107.06499] Deduplicating Training Data Makes Language Models Better
https://arxiv.org/abs/2107.06499

[6] [85] [86] Data Statements for Natural Language Processing: Toward Mitigating System Bias and Enabling Better Science - ACL Anthology
https://aclanthology.org/Q18-1041/

[7] [73] NeurIPS Poster The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data Only
https://neurips.cc/virtual/2023/poster/73487

[8] [39] [40] [74] [75] [76] FineWeb2: One Pipeline to Scale Them All — Adapting Pre-Training Data Processing to Every Language
https://arxiv.org/html/2506.20920v1

9 30 31 32 43 44 45 46 47 48 50 68 69 [2107.06499] Deduplicating Training Data Makes Language Models Better
https://ar5iv.labs.arxiv.org/html/2107.06499

10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 41 42 51 52 53 54 55 72 Large language model data pipelines and Common Crawl (WARC/WAT/WET) | Christian S. Perone
https://blog.christianperone.com/2023/06/appreciating-llms-data-pipelines/

34 35 36 37 38 87 Technical Deep-Dive: Curating Our Way to a State-of-the-Art Text Dataset
https://blog.datologyai.com/technical-deep-dive-curating-our-way-to-a-state-of-the-art-text-dataset/

56 57 61 63 64 65 MinHash - Fast Jaccard Similarity at Scale
https://arpitbhayani.me/blogs/jaccard-minhash/

58 59 60 62 Illustrated Guide to Min Hashing
https://giorgi.tech/blog/minhashing/

66 [PDF] Detecting Near-Duplicates for Web Crawling - Google Research
https://research.google.com/pubs/archive/33026.pdf

67 SimHash - Wikipedia
https://en.wikipedia.org/wiki/SimHash

70 Near Duplicate Detection Using Simhash - Sumon Biswas
https://sumonbis.github.io/academic-project/simhash/

71 Finding near-duplicate web pages: A large-scale evaluation of ...
https://www.researchgate.net/publication/221299744_Finding_near-duplicate_web_pages_A_large-scale_evaluation_of_algorithms

77 78 SlimPajama: A 627B token, cleaned and deduplicated version of ...
https://www.cerebras.ai/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama